TEC | Tecnológico de Costa Rica

PY01 - Análisis Léxico y Sintáctico

IC5701 – Compiladores e Intérpretes

Estudiantes

David Calvo García
Erin Camacho González

Profesor
Allan Rodríguez Dávila

I Semestre

29 de Abril 2026

Tabla de contenido

Manual de usuario 3

Pruebas de funcionalidad 4

Descripción del problema 9

Diseño del programa 9

Librerías usadas 10

Análisis de resultados 11

Justificación de toma de decisiones 13

Bitácora 14

Manual de usuario

Objetivo del sistema

- El sistema permite analizar un archivo fuente escrito en un lenguaje definido, realizando análisis léxico y sintáctico, generando tokens, errores y una tabla de símbolos.

Clonar el repositorio

Para obtener el proyecto desde GitHub, ejecutar:

- git clone https://github.com/Dcalvog25/PY01-CI.git

Requisitos

- Java instalado (JDK)
- Archivos generados por JFlex y CUP
- Consola o terminal

Compilación

Generar el lexer (JFlex):

- java -jar .\lib\jflex-full-1.9.1.jar .\src\scanner.flex

Generar el parser (CUP):

- java -jar lib/java-cup-11b.jar -parser parser -symbols sym -destdir src src/parser.cup

Compilar el proyecto:

- javac -cp "lib/java-cup-11b.jar;lib/java-cup-11b-runtime.jar" -d bin src/*.java

Ejecución

- java -cp "bin;lib/java-cup-11b.jar;lib/java-cup-11b-runtime.jar" Main

Uso

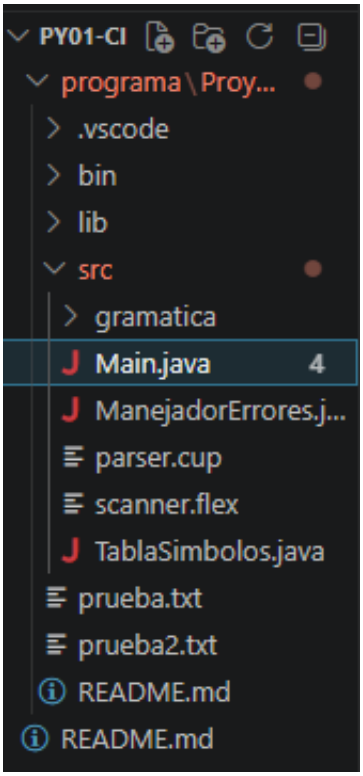
- Ejecutar el programa
- Ingresar el nombre del archivo (ej: prueba.txt)
- El sistema realizará el análisis

Archivos generados

- tokens.txt → lista de tokens
- errores.txt → reporte de errores
- tabla_simbolos.txt → tabla de símbolos

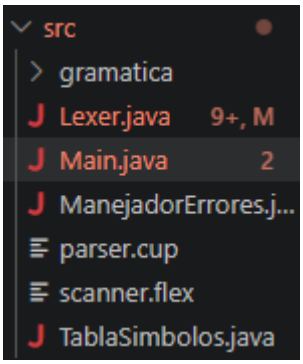
Pruebas de funcionalidad

Primeramente, en la siguiente imagen se puede observar los archivos en la carpeta si aún no se ha creado el parser y el lexer.



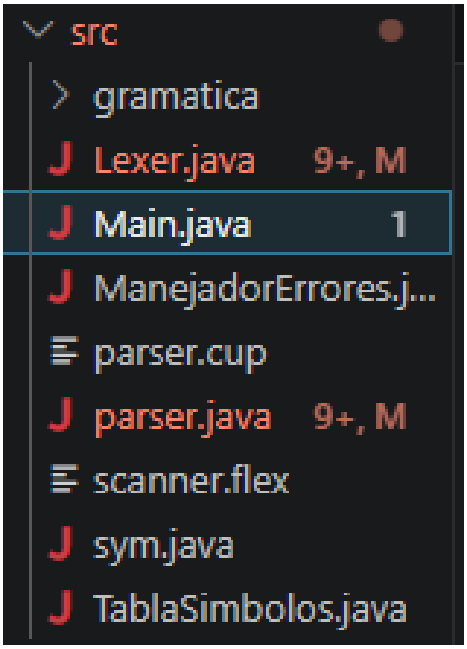
A continuación, se debe de ingresar a programa y luego a Proyecto01-Compi. Una vez ahí, se debe de escribir el siguiente comando para la creación del lexer. Esto toma el código del scanner.flex y crea el lexer.java.

```
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI> cd programa
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa> cd Proyecto01-Compi
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi> java -jar .\lib\jflex-full-1.9.1.jar .\src\scanner.flex
Reading ".\src\scanner.flex"
Constructing NFA : 410 states in NFA
Converting NFA to DFA :
.........................................................................................................................................
.................................
201 states before minimization, 181 states in minimized DFA
Writing code to ".\src\Lexer.java"
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi>
```



Seguido a esto, se escribe el comando correspondiente para la creación del cup. Con esto se estaría creando el archivo parser.java que es donde se realiza todo el análisis sintáctico con el uso de la gramática.

```
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi> java -jar .\lib\java-cup-11b.jar -parser parser -symbols sym .\src\
parser.cup
------- CUP v0.11b 20160615 (GIT 4ac7450) Parser Generation Summary -------
  0 errors and 0 warnings
  57 terminals, 72 non-terminals, and 147 productions declared,
  producing 311 unique parse states.
  0 terminals declared but not used.
  0 non-terminals declared but not used.
  0 productions never reduced.
  0 conflicts detected (0 expected).
  Code written to "parser.java", and "sym.java".
---------------------------------------------------- (CUP v0.11b 20160615 (GIT 4ac7450))
```



Luego se inserta el siguiente comando para la compilación del programa, si no existen errores se puede continuar.

```
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi> javac -cp ".\lib\java-cup-11b.jar;.\lib\java-cup-11b-runtime.jar" -
d bin src\*.java
Note: src\parser.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
```

Por último, para ejecutar el programa se debe de ingresar el siguiente comando que inicia todo. En seguida, se le pide al usuario el nombre del archivo que desea compilar.

```
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi> java -cp "bin;lib/java-cup-11b.jar;lib/java-cup-11b-runtime.jar" Ma
in
Ingrese el nombre del archivo:
```

Al ingresar el nombre del archivo, el programa inicia el análisis léxico y sintáctico del código proporcionado. En el caso de que existieran errores, estos se mostrarán en pantalla. Además se creará un archivo con los errores, otro con los tokens y otro con la tabla de símbolos.

Consola:

```
PS C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi> java -cp "bin;lib/java-cup-11b.jar;lib/java-cup-11b-runtime.jar" Ma
in
Ingrese el nombre del archivo: prueba2.txt
Error Sintáctico
  Línea   : 4
  Columna : 14
  Mensaje : Token inesperado: FIN_EXPR

Error Sintáctico
  Línea   : 13
  Columna : 14
  Mensaje : Token inesperado: FIN_EXPR

Error Sintáctico
  Línea   : 21
  Columna : 10
  Mensaje : Token inesperado: FIN_EXPR

Error Sintáctico
  Línea   : 39
  Columna : 19
  Mensaje : Token inesperado: FINAL_PAREN

Análisis finalizado con errores
```

Archivos creados:

```
errores.txt        U
prueba.txt
prueba2.txt        M
README.md
tabla_simbolos.... U
tokens.txt         U
```

errores.txt:

```
programa > Proyecto01-Compi > errores.txt
1   ====================================================
2                    REPORTE DE ERRORES
3   ====================================================
4   Tipo         Línea    Columna  Mensaje
5   ----------------------------------------------------
6   Sintáctico   4        14       Token inesperado: FIN_EXPR
7   Sintáctico   13       14       Token inesperado: FIN_EXPR
8   Sintáctico   21       10       Token inesperado: FIN_EXPR
9   Sintáctico   39       19       Token inesperado: FINAL_PAREN
10  ----------------------------------------------------
11  Total de Errores: 4
12  ====================================================
13
```

tokens.txt:

programa > Proyecto01-Compi > tokens.txt

```
INT int 1   1
SEPARADOR   ~   1   4
ID  sumar   1   5
INICIO_PAREN    <|  1   10
INT int 1   12
SEPARADOR   ~   1   15
ID  a   1   16
COMA    ,   1   17
INT int 1   18
SEPARADOR   ~   1   21
ID  b   1   22
FINAL_PAREN |>  1   23
INICIO_BLOQUE   |:  2   1
INT int 3   1
SEPARADOR   ~   3   4
ID  resultado   3   5
FIN_EXPR    !   3   14
ID  resultado   4   1
ASIGNAR <-  4   10
ID  a   4   12
SUMA    +   4   13
FIN_EXPR    !   4   14
FLOAT   float   5   1
SEPARADOR   ~   5   6
ID  temp    5   7
FIN_EXPR    !   5   11
RETURN  return  6   1
SEPARADOR   ~   6   7
ID  resultado   6   8
FIN_EXPR    !   6   17
FINAL_BLOQUE    :|  7   1
FLOAT   float   9   1
```

tabla_simbolos.txt:

programa > Proyecto01-Compi > tabla_simbolos.txt

```
==========================================================================================
                                    TABLA DE SÍMBOLOS
==========================================================================================
Total de scopes: 7

------------------------------------------------------------------------------------------
SCOPE: comparar1_global  (padre: global)
Total de símbolos: 3
------------------------------------------------------------------------------------------
Id                    Tipo        Categoría      Línea    Columna  Valor          Parámetros
------------------------------------------------------------------------------------------
x                     int         parametro      16       20       -              -
y                     int         parametro      16       26       -              -
bandera               bool        variable       18       13       -              -

------------------------------------------------------------------------------------------
SCOPE: else1_comparar1_global  (padre: comparar1_global)
Total de símbolos: 0
------------------------------------------------------------------------------------------
  (sin símbolos registrados)

------------------------------------------------------------------------------------------
SCOPE: global  (padre: -)
Total de símbolos: 3
------------------------------------------------------------------------------------------
Id                    Tipo        Categoría      Línea    Columna  Valor          Parámetros
------------------------------------------------------------------------------------------
sumar                 int         funcion        1        5        -              -
promedio              float       funcion        9        7        -              -
comparar              bool        funcion        16       6        -              -

------------------------------------------------------------------------------------------
```

programa > Proyecto01-Compi > tabla_simbolos.txt

```
SCOPE: main  (padre: global)
Total de símbolos: 4
------------------------------------------------------------------------------------------
Id                    Tipo        Categoría      Línea    Columna  Valor          Parámetros
------------------------------------------------------------------------------------------
num1                  int         variable       32       13       -              -
num2                  int         variable       33       12       -              -
prom                  float       variable       34       11       -              -
ok                    bool        variable       35       8        -              -

------------------------------------------------------------------------------------------
SCOPE: promedio1_global  (padre: global)
Total de símbolos: 3
------------------------------------------------------------------------------------------
Id                    Tipo        Categoría      Línea    Columna  Valor          Parámetros
------------------------------------------------------------------------------------------
n1                    float       parametro      9        23       -              -
n2                    float       parametro      9        32       -              -
total                 float       variable       11       12       -              -

------------------------------------------------------------------------------------------
SCOPE: sumar1_global  (padre: global)
Total de símbolos: 4
------------------------------------------------------------------------------------------
Id                    Tipo        Categoría      Línea    Columna  Valor          Parámetros
------------------------------------------------------------------------------------------
a                     int         parametro      1        16       -              -
b                     int         parametro      1        22       -              -
resultado             int         variable       3        14       -              -
temp                  float       variable       5        11       -              -
```

En el caso de que el archivo proporcionado no tuviera errores, se mostrará en consola que la compilación fue exitosa. El archivo de errores estaría vacío, igualmente se puede observar los tokens y la tabla de símbolos completa en sus respectivos archivos.

```
in C:\Users\erinc\OneDrive - Estudiantes ITCR\Escritorio\PY01-CI\programa\Proyecto01-Compi>
Ingrese el nombre del archivo: prueba2.txt
Compilación exitosa
```

Descripción del problema

El problema consiste en construir un analizador para un lenguaje de programación definido mediante una gramática que se realizó en la tarea 01. Este analizador debe ser capaz de leer un archivo fuente, identificar los elementos válidos del lenguaje y determinar si la estructura del programa cumple con las reglas establecidas.

Para resolverlo, se implementaron dos etapas principales. La primera corresponde al análisis léxico, donde el programa divide el código fuente en tokens, como palabras reservadas, identificadores, tipos de datos, operadores, literales y símbolos especiales. La segunda etapa corresponde al análisis sintáctico, donde esos tokens se revisan según la gramática para comprobar si forman instrucciones válidas.

Además, el proyecto incluye manejo de errores y generación de una tabla de símbolos. Esto permite no solo detectar problemas en el código, sino también registrar información importante sobre funciones, variables, parámetros, arreglos y scopes. De esta manera, el sistema no se limita a decir si el archivo es válido o no, sino que también produce reportes útiles para comprender cómo fue interpretado el programa fuente.

Diseño del programa

El programa se diseñó separando responsabilidades en diferentes componentes. Esta decisión permite que cada parte tenga una función clara y sea más fácil de mantener, probar y corregir.

El primer componente es el Lexer, implementado con JFlex. Su objetivo es leer carácter por carácter el archivo fuente y reconocer patrones válidos. Por ejemplo,

identifica palabras reservadas como if, while, return, tipos como int, float, bool, y símbolos como <-, <|, |>, |:, :| y !.

El segundo componente es el Parser, implementado con CUP. Este recibe los tokens generados por el Lexer y los organiza según la gramática del lenguaje. Aquí se definen reglas para reconocer funciones, el método principal __main__, bloques, sentencias, asignaciones, expresiones, estructuras de control, llamadas a funciones y arreglos.

El tercer componente es la Tabla de Símbolos. Esta se utiliza para guardar los identificadores encontrados durante el análisis, clasificados por scope. Por ejemplo, las funciones se almacenan en el scope global, mientras que los parámetros y variables locales se almacenan dentro del scope correspondiente a cada función o bloque.

También se implementó un Manejador de Errores, encargado de centralizar los errores léxicos y sintácticos. Esta clase permite generar un archivo errores.txt con línea, columna, tipo de error y mensaje.

Entre las principales decisiones de diseño se encuentran:

- Usar archivos separados para lexer, parser, tabla de símbolos, errores y clase principal.
- Generar reportes externos para facilitar la revisión.
- Usar HashMap para organizar los scopes de la tabla de símbolos.
- Usar Stack para controlar la entrada y salida de scopes anidados.
- Aplicar recuperación de errores en el parser para evitar que el análisis se detenga ante el primer problema.

Librerías usadas

El proyecto utiliza librerías estándar de Java y librerías necesarias para integrar JFlex y CUP.

La librería java.io.* se utiliza para trabajar con archivos. Gracias a ella, el programa puede leer el archivo fuente ingresado por el usuario y generar archivos de salida como tokens.txt, errores.txt y tabla_simbolos.txt.

La librería java.util.* se utiliza para manejar estructuras de datos como listas, pilas, mapas y conjuntos. En este proyecto se usa principalmente para almacenar errores, manejar scopes y organizar la tabla de símbolos.

La librería java_cup.runtime.* permite que el analizador léxico y el sintáctico se comuniquen. El Lexer devuelve objetos Symbol, que son interpretados por el Parser durante el análisis sintáctico.

También se utilizan los archivos .jar de CUP y JFlex. JFlex permite generar automáticamente la clase Lexer.java a partir del archivo scanner.flex, mientras que CUP genera parser.java y sym.java a partir del archivo parser.cup.

Análisis de resultados

Objetivo	**Porcentaje de Realización**	**Justificación**
Implementar el análisis léxico	100%	Se logró identificar correctamente los tokens del lenguaje, incluyendo palabras reservadas, identificadores, tipos de datos, operadores y símbolos especiales. Además, los tokens se registran en el archivo tokens.txt con su respectiva posición.
Detectar errores léxicos	100%	El sistema detecta caracteres inválidos y los registra correctamente en errores.txt indicando línea y columna. El análisis continúa sin detenerse, permitiendo encontrar múltiples errores en una sola ejecución.

Implementar el análisis sintáctico	100%	Se desarrolló una gramática funcional con CUP capaz de reconocer estructuras como funciones, main, declaraciones, asignaciones, expresiones, estructuras de control y llamadas a funciones.
Manejo de expresiones con precedencia	100%	Se implementó correctamente la jerarquía de expresiones (lógicas, relacionales, aritméticas, unarias), permitiendo evaluar correctamente la precedencia de operadores y evitando ambigüedades en la gramática.
Manejo de errores sintácticos	95%	Se implementó recuperación de errores mediante modo pánico, permitiendo continuar el análisis en la mayoría de los casos. Sin embargo, errores estructurales fuertes (como desbalance de bloques) pueden provocar finalización anticipada.
Generación de tabla de símbolos	100%	Se logró almacenar correctamente variables, funciones, parámetros y arreglos, organizados por scopes. La información se exporta a tabla_simbolos.txt de forma clara y estructurada.
Manejo de scopes	100%	Se implementó correctamente el manejo de scopes utilizando una pila, permitiendo diferenciar entre

		variables globales, locales y parámetros dentro de funciones y estructuras de control.
Generación de archivos de salida	100%	El sistema genera correctamente los archivos tokens.txt, errores.txt y tabla_simbolos.txt, facilitando la revisión del análisis realizado.
Integración de componentes (Lexer + Parser + Tabla)	100%	Se logró una integración completa entre los módulos del sistema, permitiendo que el lexer, parser y tabla de símbolos trabajen de forma coordinada durante el análisis.
Robustez ante entradas con errores	95%	El sistema logra manejar múltiples errores en una misma ejecución. Sin embargo, en casos donde la estructura del programa se rompe completamente, el parser puede no recuperarse completamente hasta el final.

Justificación de toma de decisiones

Una decisión importante fue dividir las expresiones en varios niveles: lógica, conjunción, negación, relacional, aritmética, término, factor, potencia y unarios. Esta separación permite manejar mejor la precedencia de operadores y evitar ambigüedades en el parser.

Por ejemplo, las operaciones de multiplicación y división deben tener mayor prioridad que la suma y la resta. Asimismo, las expresiones lógicas deben evaluarse después de las relacionales y aritméticas. Por eso se diseñó una jerarquía gramatical que permite procesar expresiones de forma ordenada.

Otra decisión fue aplicar recuperación de errores en modo pánico. Para esto se usaron reglas con error en CUP, especialmente en listas de sentencias y bloques. La idea es que cuando el parser detecta una instrucción inválida, pueda saltar hasta un punto seguro como el fin de expresión ! y continuar analizando las siguientes instrucciones.

También se decidió manejar los scopes mediante una pila. Esto permite entrar a un nuevo scope cuando se reconoce una función, un if, un else, un do-while o un switch, y salir correctamente cuando finaliza el bloque. Esta estructura facilita representar mejor el alcance de las variables.

Finalmente, se decidió generar reportes en archivos externos porque facilita la revisión del proyecto. En lugar de depender únicamente de la consola, el sistema produce archivos organizados que pueden ser incluidos como evidencia en la documentación.

Bitácora

https://github.com/Dcalvog25/PY01-CI.git